

实验三：定位

一 实验目标

1. 了解像素坐标，机器人坐标，世界坐标之间的坐标变换
2. 了解摄像头参数和标定参数的方法
3. 了解如何使用 AprilTag 定位

二 考核要求

【考核 1】定位

【实验说明】在比赛场地上进行验收。验收时，助教将机器人随机放置在其中一个点位上，并确保机器人视野中存在可供识别的 AprilTag。机器人需要通过 Tag 确定自身位置，向控制终端返回自身坐标，并判断自己所处的点位序号。验收示范可参考网络学堂“[实验三验收视频.mp4](#)”。

【验收要求】

返回坐标与参考坐标之间的绝对误差不超过 15cm

【考核 2】机器人身体朝向判断

【实验说明】为了让机器人沿规划好的路线前进，我们不仅需要确定机器人的位置，还需要知道机器人身体的朝向。本次实验在比赛场地上进行验收。验收时，助教将机器人放在场地上的随机位置，并确保机器人视野中存在可供识别的 AprilTag。机器人需要通过 Tag 确定摄像头的朝向，并向控制终端返回自己的朝向。验收示范可参考网络学堂“[实验三验收视频.mp4](#)”。

【验收要求】

机器人只需要正确判断自身朝向靠近 x 轴正负/y 轴正负方向中的哪一个即可

三 实验指导

3.1 坐标变换

3.1.1 四个坐标系

从物理世界到照片的过程会经过下面四个坐标系，如图1所示：

- 世界坐标：空间直角坐标，与地表相对静止，记作 $r_w = [x_w \ y_w \ z_w]^T$
- 相机坐标：空间直角坐标，原点位于透视点， z 轴与相机光轴重合，正方向与光线方向相反。相机坐标与相机相对静止，记作 $r_c = [x_c \ y_c \ z_c]^T$
- 图像坐标：投影面上的直角坐标，原点位于光轴与投影面的交点，坐标轴与相机坐标平行，记作 $r = [x \ y]^T$
- 像素坐标：数字图像的坐标，左上角为原点， u 轴向右， v 轴向下，记作 $p = [u \ v]^T$

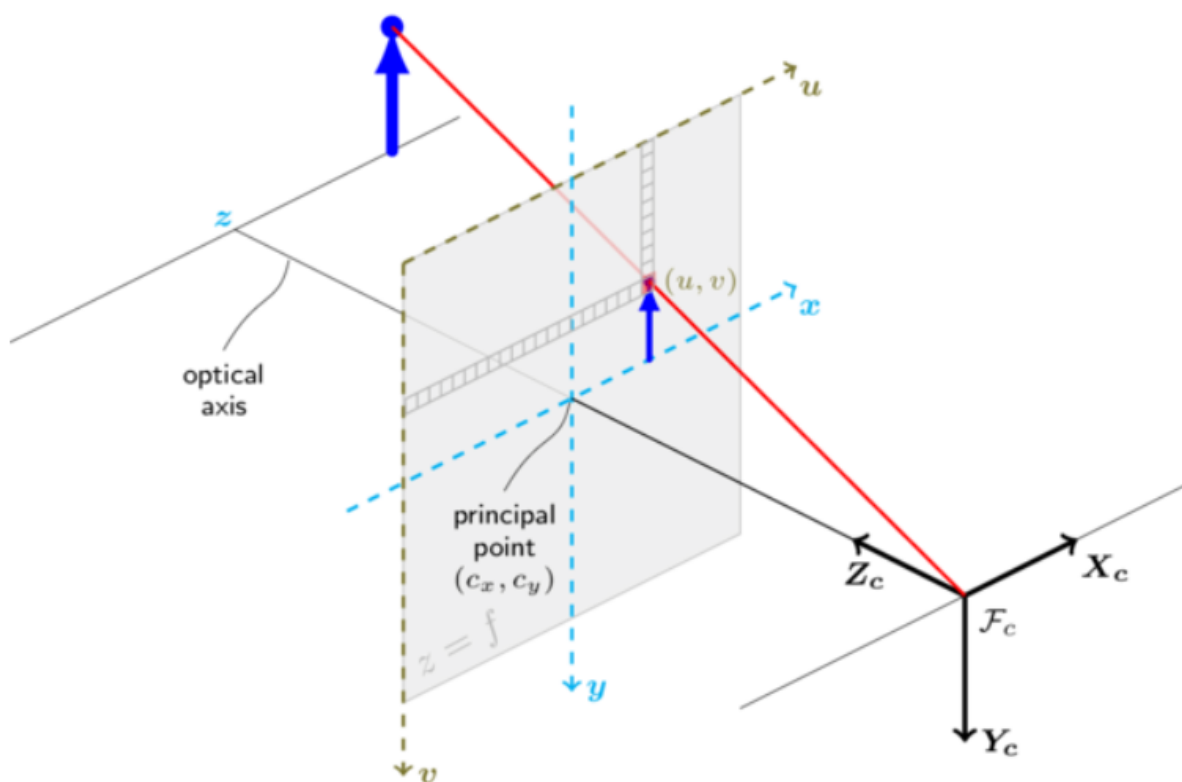


图 1: 四个坐标系示意图

3.1.2 坐标系的变换关系

世界坐标到相机坐标

世界坐标与相机坐标之间是平移 + 旋转关系：

$$r_c = Rr_w + t \quad (1)$$

将其写为分块矩阵乘法形式：

$$\begin{bmatrix} r_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} r_w \\ 1 \end{bmatrix} \quad (2)$$

相机坐标到图像坐标

相机坐标与图像坐标之间是点透视关系，根据简单的几何学，由位似关系可知：

$$\frac{x}{x_c} = \frac{y}{y_c} = \frac{f}{z_c} \quad (3)$$

其中 f 是透视点到投影面的距离，也可以理解为透镜组的等效焦距。将其写为矩阵乘法形式：

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \frac{1}{z_c} \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} \quad (4)$$

图像坐标到像素坐标

通常约定图像坐标和像素坐标是平移 + 量化关系，如果忽略量化带来的离散性，那么可以将量化视作伸缩：

$$\begin{aligned} u &= \frac{x}{d_x} + u_0 \\ v &= \frac{y}{d_y} + v_0 \end{aligned} \quad (5)$$

其中 d_x, d_y 代表一个像素的大小。将其写为矩阵乘法形式：

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{d_x} & 0 & u_0 \\ 0 & \frac{1}{d_y} & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (6)$$

世界坐标到像素坐标

综合上面的结果可以得到：

$$\begin{bmatrix} p \\ 1 \end{bmatrix} = \frac{1}{z_c} \begin{bmatrix} C & 0 \end{bmatrix} \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} r_w \\ 1 \end{bmatrix} \quad (7)$$

其中 $C = \begin{bmatrix} f_x & 0 & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}$, $f_x = \frac{f}{d_x}$, $f_y = \frac{f}{d_y}$ 。称 C 为相机的内部参数, R 和 t 为相机的外部参数。

畸变现象

众所周知,理想透镜成像理论只对傍轴光线近似成立,对于大角度的光线,透镜通常会产生畸变,如图2所示。因此需要对图像坐标进行修正:

$$\begin{aligned} x' &= x(1 + k_1r^2 + k_2r^4) + p_1(r^2 + 2x^2) + 2p_2xy \\ y' &= y(1 + k_1r^2 + k_2r^4) + p_2(r^2 + 2y^2) + 2p_1xy \end{aligned} \quad (8)$$

记 $D = \begin{bmatrix} k_1 & k_2 & p_1 & p_2 \end{bmatrix}$, 称为相机的畸变系数。

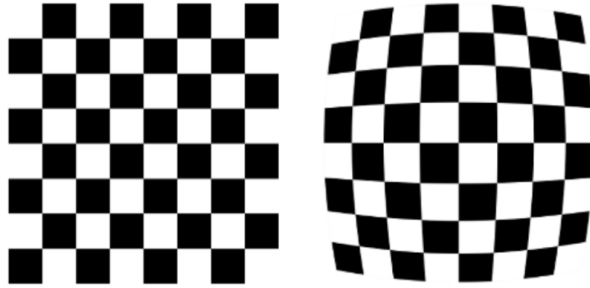


图 2: 畸变示意图

3.2 参数求解

如果已知若干组像素坐标和世界坐标的对应关系,那么可以解出相机的外部参数、内部参数和畸变矩阵。相机的外部参数会随相机的位置改变,相机的内部参数和畸变系数是固定不变的。

3.2.1 相机标定——求解内部参数和畸变矩阵

利用标定板和 MATLAB 的 Camera Calibrator,可参考视频“[实验三相机标定.mp4](#)”。

拍摄照片

首先需要打印一张标定板图像。推荐一个生成相机标定板文件(pdf)的网站:<https://calib.io/pages/camera-calibration-pattern-generator>。调用机器人的摄像头,通过调整标定板或摄像机的方向,为标定物拍摄一组不同方向的照片,获取至少 10 张照片并保存,如图3所示,然后导出到本地电脑中。

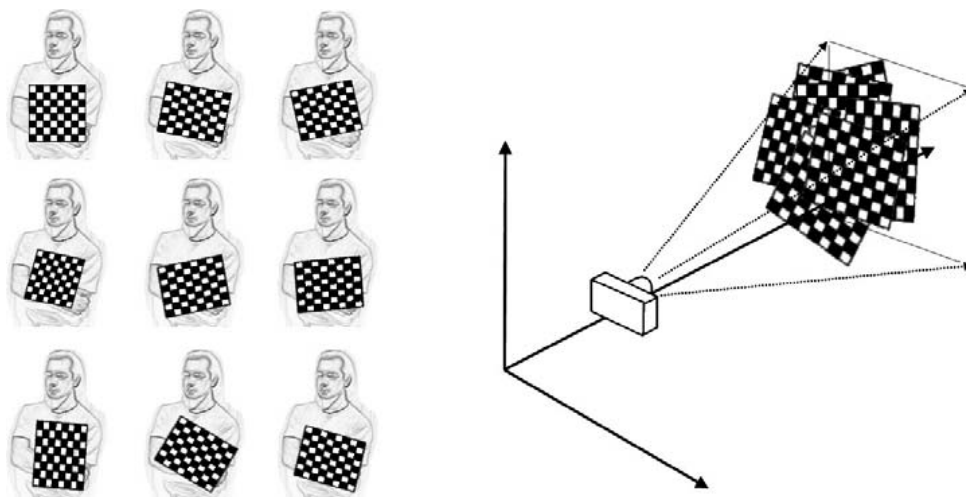


图 3

MATLAB 标定

打开 Matlab, 在 Matlab 中找到 APP, 点击进入, 找到 Camera Calibrator 工具箱(没有就下载), 如图4所示。

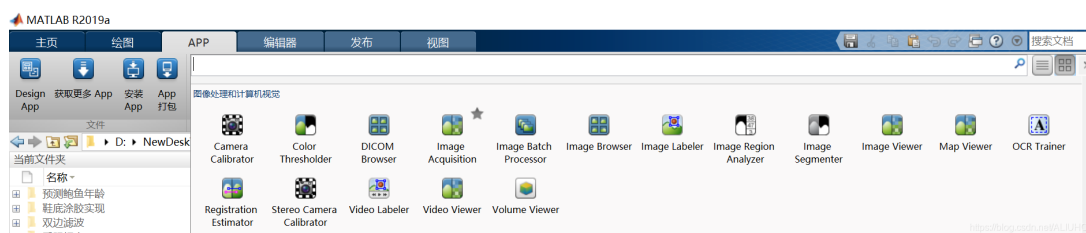


图 4

进入工具箱, 点击 Add Images, 如图5所示。

将所有图片添加进去(这里的 25 是指棋盘中正方形的边长大小, 可以根据自己生成的棋盘方格边长调整), 如图6所示。

然后点击 Calibrate 按钮, 即完成标定, 如图7所示。

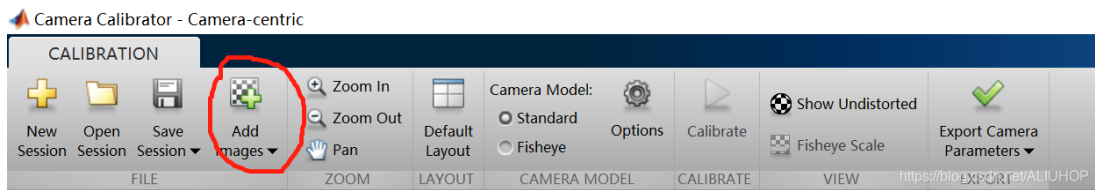


图 5



图 6

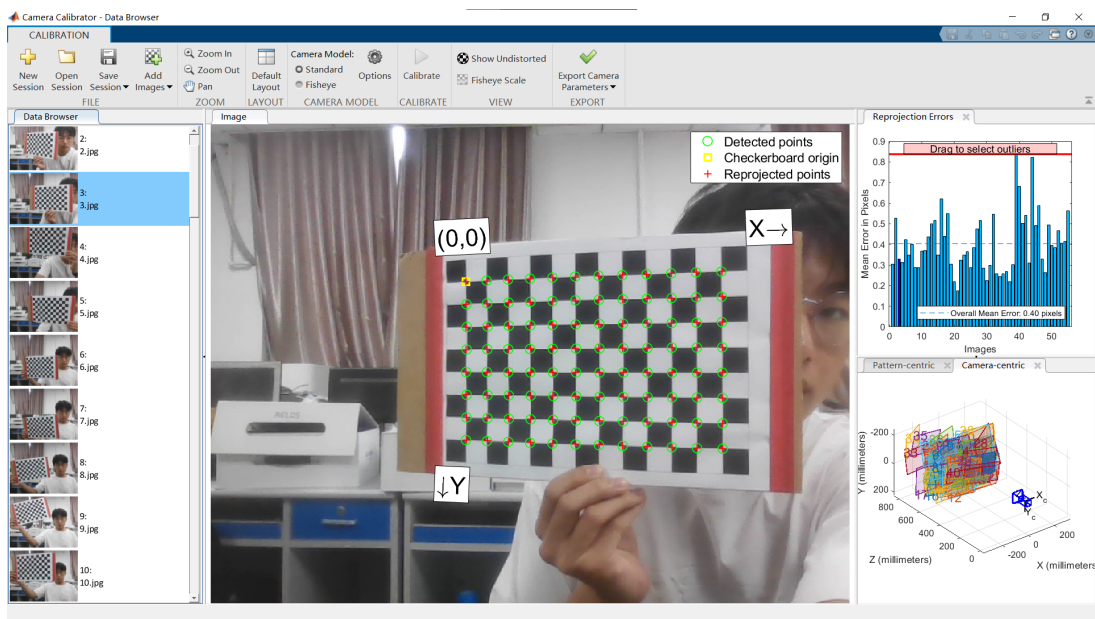


图 7

导出参数

点击 Export Camera Parameters, 如图8所示。

可在 matlab 工作空间里可以看到相机参数的属性, 如图9所示。其中的 IntrinsicMatrix 是相机的内部参数, $\begin{bmatrix} \text{RadialDistortion} & \text{TangentialDistortion} \end{bmatrix}$ 组成相机的畸变系数。

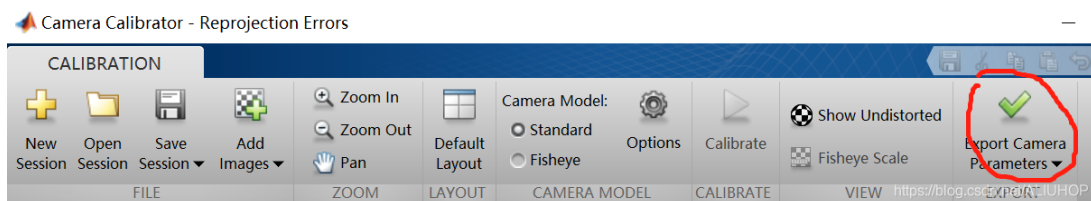


图 8

属性	值
ImageSize	[480,640]
RadialDistortion	[-0.1467,0.3581]
TangentialDistortion	[0,0]
WorldPoints	96x2 double
WorldUnits	'millimeters'
☒ EstimateSkew	0
NumRadialDistortionCoefficients	2
☒ EstimateTangentialDistortion	0
TranslationVectors	55x3 double
ReprojectionErrors	96x2x55 double
RotationVectors	55x3 double
NumPatterns	55
☒ Intrinsics	1x1 cameraIntr...
IntrinsicMatrix	[624.1770,0,0;...
FocalLength	[624.1770,629....
PrincipalPoint	[324.3789,238....
Skew	0
MeanReprojectionError	0.4028
ReprojectedPoints	96x2x55 double
RotationMatrices	3x3x55 double

图 9

3.2.2 Perspective-n-Point(PnP) 问题——求解外部参数

可以利用 opencv 的 solvePnP 和 Rodrigues 函数来求解 PnP 问题，下面给出函数的简要说明，更加详细的内容可以参考 opencv 的官方文档。

solvePnP

```
cv.solvePnP(objectPoints, imagePoints, cameraMatrix, distCoeffs
[, rvec[, tvec[, useExtrinsicGuess[, flags]]]]) -> retval,
rvec, tvec
```

输入：

- objectPoints: 形状为 $N \times 3$ 的 numpy 数组，表示 N 个点的世界坐标
- imagePoints: 形状为 $N \times 2$ 的 numpy 数组，表示 N 个点的像素坐标
- cameraMatrix: 形状为 3×3 的 numpy 数组，表示相机的内部参数
- distCoeffs: 长度为 4 的 numpy 数组，表示相机的畸变系数

输出：

- retval: 是否计算成功
- tvec: 平移向量，即公式7中的 t
- rvec: 旋转向量，需要通过 Rodrigues 函数转换为公式7中的 R

备注：

- objectPoints 和 imagePoint 形状的 N 必须相同，并且 $N \geq 4$
- 本实验中 objectPoints 和 imagePoints 选取了场地上 AprilTag 的各个角点，objectPoints 可以由自定义函数 load_tag_pos 获得，详见第3.4节，imagePoints 可以由图像识别结果得到，详见实验二。

Rodrigues

```
cv.Rodrigues(src[, dst[, jacobian]]) -> dst, jacobian
```

输入：

- src: 长度为 3 的 numpy 数组，表示旋转向量，即 solvePnP 的输出 rvec

输出：

- dst: 形状为 3×3 的 numpy 数组，表示旋转矩阵，即公式7中的 R

3.3 方位估计

相机的方位与外部参数之间存在一一对应关系，如果已知外部参数，那么可以得到相机的方位。具体来说，相机坐标系的原点即是相机的位置，它的世界坐标为 $-R^{-1}t$ ，相机坐标系的 z 轴单位向量即是相机的方向，它的世界坐标为 $R^{-1} \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$ 。

3.4 场地上 AprilTag 的世界坐标

文件 `load_tags.py` 中定义了函数 `load_tag_pos`，它返回一个字典，包含所有 AprilTag 的世界坐标。每个 AprilTag 的世界坐标由一个形状为 4×3 的 numpy 数组表示，数组各行依次是 AprilTag 左上角、右上角、右下角、左下角的坐标，这个顺序与 `apriltag.Detector.detect` 识别结果的顺序是一致的。

3.5 scp 命令

机器人和电脑之间传输文件可以使用 scp 命令：

1. 查找机器人 ip 地址，bash 下使用命令，终端中就会给出当前网络中机器人的 IPV4 地址

```
hostname -I
```

2. 文件从机器人到电脑

```
scp remote_username@remote_ip:remote_file_path  
local_file_path
```

文件夹从机器人到电脑

```
scp -r remote_username@remote_ip:remote_folder_path  
local_folder_path
```

3. 文件从电脑到机器人

```
scp local_file_path remote_username@remote_ip:  
remote_file_path
```

文件夹从电脑到机器人

```
scp -r local_folder_path remote_username@remote_ip:  
remote_folder_path
```

PS：如果你有在机器人上限制访问端口，可在 -r 后加上 -P port 来通过指定端口进行文件传输。也可以不使用 scp，直接在 jupyter notebook 上逐个下载文件。